

FORMATION EMBARQUÉE

Module 00 — Fondamentaux

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 00 — Fondamentaux : ce que tout programmeur embarqué doit savoir

Objectif : comprendre ce qui se passe *physiquement* quand un programme s'exécute. Sans ça, les pointeurs, les registres et le VHDL resteront de la magie noire.

1. Le binaire et l'hexadécimal

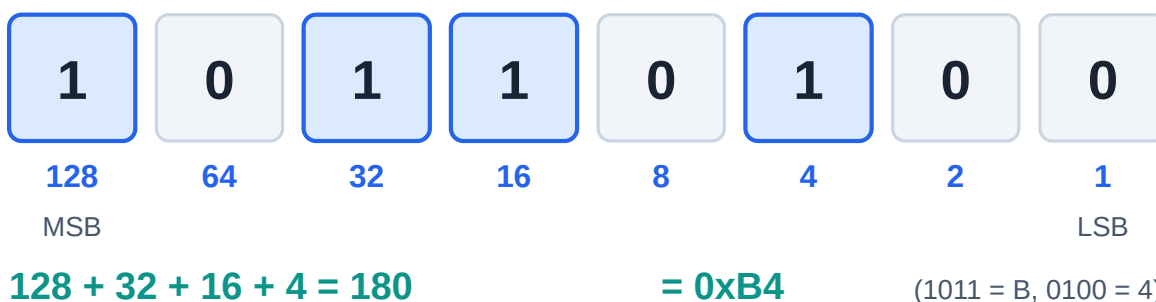
Un ordinateur ne connaît que deux états : tension présente (1) ou absente (0). Tout — nombres, textes, images, instructions — est codé en binaire.

1.1 Compter en binaire

Chaque position vaut une puissance de 2 (comme chaque position vaut une puissance de 10 en décimal) :

```
binaire : 1 0 1 1
poids   : 8 4 2 1    → 8 + 0 + 2 + 1 = 11 en décimal
```

Nombre binaire 1011 0100 sur 8 bits



Poids des bits sur un octet et conversion en hexadécimal

Décimal	Binaire (4 bits)	Hexadécimal
0	0000	0x0
5	0101	0x5
10	1010	0xA
15	1111	0xF
255	1111 1111	0xFF

1.2 L'hexadécimal : le binaire pour humains

4 bits = 1 chiffre hexa (0–F). On écrit `0xFF` plutôt que `11111111`. En embarqué, **tout** se lit en hexa : adresses mémoire (`0x40021000`), valeurs de registres (`0x0000000F`), trames de communication.

1.3 Vocabulaire

- **bit** : un 0 ou un 1.
- **octet (byte)** : 8 bits. Valeurs de 0 à 255 (`0x00` à `0xFF`).
- **mot (word)** : dépend du processeur — 16, 32 ou 64 bits.
- **LSB / MSB** : bit de poids faible / fort.
- **endianness** : ordre des octets en mémoire. *Little-endian* (x86, la plupart des ARM) = octet de poids faible en premier.

1.4 Nombres signés : le complément à deux

Pour représenter les négatifs, on utilise le complément à deux : on inverse tous les bits puis on ajoute 1.

```
+5 sur 8 bits : 0000 0101
-5 sur 8 bits : 1111 1011 (inversion → 1111 1010, +1 → 1111 1011)
```

Sur 8 bits signés : plage de -128 à +127. Sur 8 bits non signés : 0 à 255. **Piège classique** : `uint8_t x = 0; x--;` donne 255, pas -1 (débordement).

Exercices

1. Convertis 42, 100 et 200 en binaire puis en hexa.
2. Que vaut `0xB7` en décimal ? En binaire ?
3. Écris -10 en complément à deux sur 8 bits.

2. Électronique numérique de base

2.1 Les portes logiques

Toute la logique d'un processeur est faite de portes :

Porte	Symbole C	Sortie = 1 si...
NOT (NON)	<code>!</code> , <code>~</code>	l'entrée est 0
AND (ET)	<code>&&</code> , <code>&</code>	toutes les entrées sont 1
OR (OU)	<code> </code> , <code> </code>	au moins une entrée est 1
XOR (OU exclusif)	<code>^</code>	les entrées sont différentes
NAND / NOR	<code>—</code>	l'inverse de AND / OR

La NAND est « universelle » : on peut construire n'importe quel circuit avec uniquement des NAND.

2.2 Logique combinatoire vs séquentielle

- **Combinatoire** : la sortie ne dépend *que* des entrées actuelles (additionneur, multiplexeur, décodeur). Pas de mémoire.
- **Séquentielle** : la sortie dépend aussi de l'état *passé* → il y a de la **mémoire**. La brique de base est la **bascule D (D flip-flop)** : elle recopie son entrée D sur sa sortie Q *au front montant de l'horloge*.

Cette distinction est la clé du VHDL (module 04) : on y décrit soit des blocs combinatoires, soit des registres cadencés par une horloge.

2.3 L'horloge (clock)

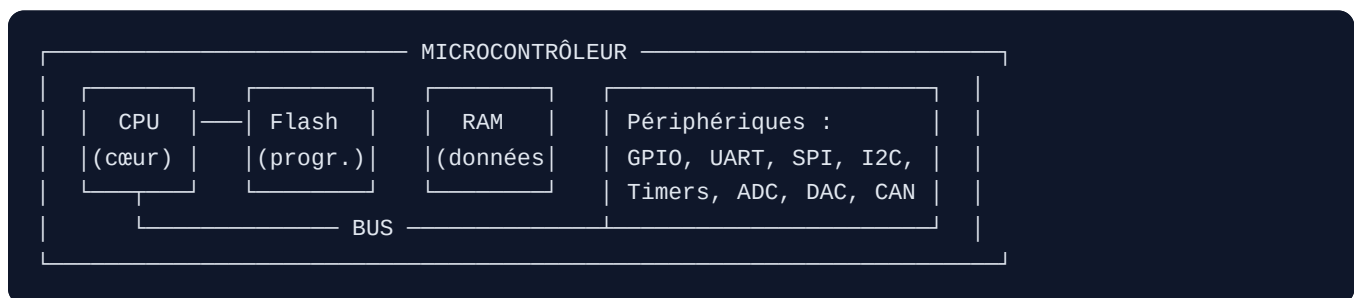
Un signal carré qui rythme tout le circuit. Un microcontrôleur à 16 MHz exécute son cycle de base 16 millions de fois par seconde. Sur FPGA, on raisonne en « que se passe-t-il à chaque front montant ? ».

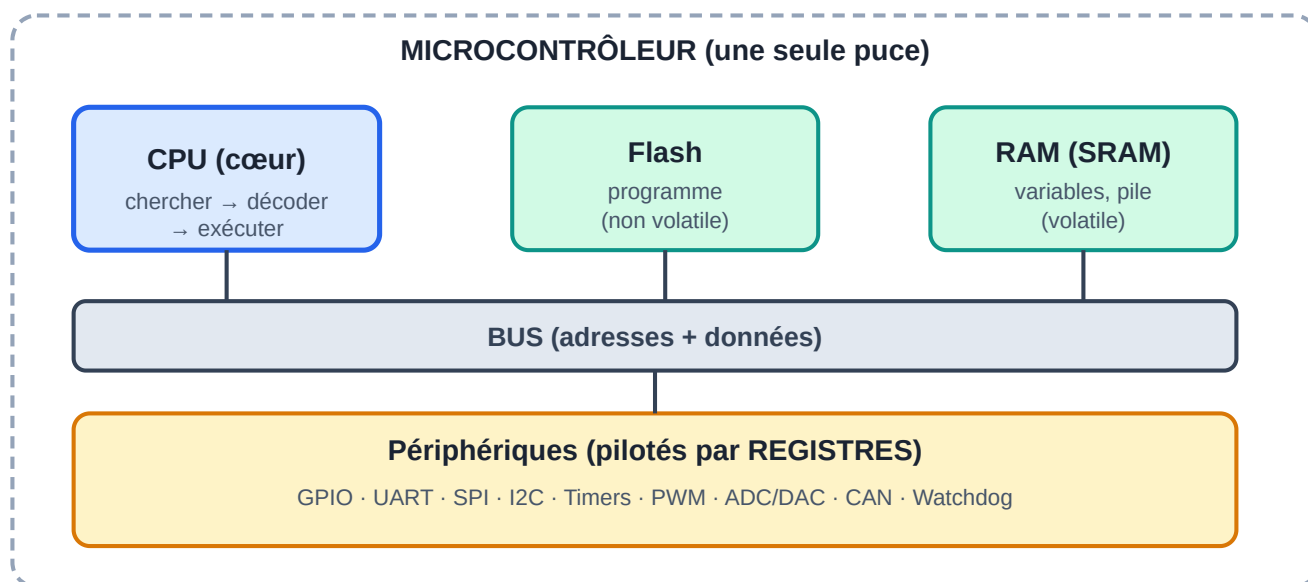
2.4 Niveaux logiques et le monde réel

- TTL 5 V (Arduino Uno) : 0 logique < 0,8 V ; 1 logique > 2 V.
- 3,3 V (STM32, ESP32, Raspberry Pi) : **ne jamais** injecter du 5 V sur une broche 3,3 V.
- **Résistance de pull-up / pull-down** : force un niveau connu quand rien ne pilote la ligne (indispensable pour les boutons et le bus I2C).
- **Rebond (bounce)** : un bouton mécanique génère des dizaines de transitions parasites pendant quelques ms. À filtrer en logiciel (attente ~20 ms) ou en matériel (condensateur).

3. Architecture d'un ordinateur / microcontrôleur

3.1 Les blocs





Blocs d'un microcontrôleur reliés par le bus

- **CPU** : exécute les instructions (chercher → décoder → exécuter).
- **Flash** : mémoire non volatile où vit le programme (survit à la coupure).
- **RAM (SRAM)** : mémoire volatile pour les variables, la pile, le tas.
- **Périphériques** : blocs matériels pilotés par des **registres**.

3.2 Microprocesseur vs microcontrôleur vs FPGA

	Microprocesseur (PC, Raspberry Pi)	Microcontrôleur (Arduino, STM32)	FPGA
Contient	CPU seul (RAM/flash externes)	CPU + RAM + flash + périphériques sur une puce	Matrice de logique reconfigurable
OS	Linux/Windows	Souvent aucun (bare-metal) ou RTOS	Aucun — c'est du matériel
Force	Puissance	Coût, temps réel, consommation	Parallélisme massif, latence nulle
Programmé en	C/C++/Python/Java	C/C++	VHDL/Verilog

3.3 Les registres : le cœur du bas niveau

Un **registre de périphérique** est une case mémoire spéciale : écrire dedans agit *physiquement* sur le matériel. Exemple (AVR/Arduino Uno) :

```
DDRB |= (1 << 5); // bit 5 de DDRB à 1 → broche PB5 en sortie
PORTB |= (1 << 5); // bit 5 de PORTB à 1 → PB5 à l'état haut → LED allumée
PORTB &= ~(1 << 5); // bit 5 à 0 → LED éteinte
```

C'est exactement ce que fait `digitalWrite()` d'Arduino sous le capot — en ~50 fois plus lent, à cause des vérifications qu'il ajoute.

3.4 Carte mémoire (memory map)

Toutes les adresses vivent dans un seul espace. Exemple typique ARM Cortex-M :

```
0x0000 0000 – Flash (code)
0x2000 0000 – SRAM (variables, pile)
0x4000 0000 – Périphériques (GPIO, UART, timers...)
0xE000 0000 – Registres système (NVIC, SysTick)
```

« Écrire à l'adresse `0x40021018` » peut vouloir dire « activer l'horloge du port GPIO C ». D'où l'importance des pointeurs en C.

3.5 Pile (stack) et tas (heap)

- **Pile** : variables locales, adresses de retour. Gérée automatiquement, rapide, mais petite (souvent quelques Ko). Le **débordement de pile** est le bug embarqué classique.
- **Tas** : allocation dynamique (`malloc` / `new`). En embarqué critique on l'évite (fragmentation, imprévisibilité) : tout est alloué statiquement.

4. Les périphériques essentiels

4.1 GPIO (General Purpose Input/Output)

Broche configurable en entrée ou sortie numérique. Modes usuels : sortie push-pull, sortie open-drain, entrée flottante, entrée avec pull-up.

4.2 Timers / compteurs

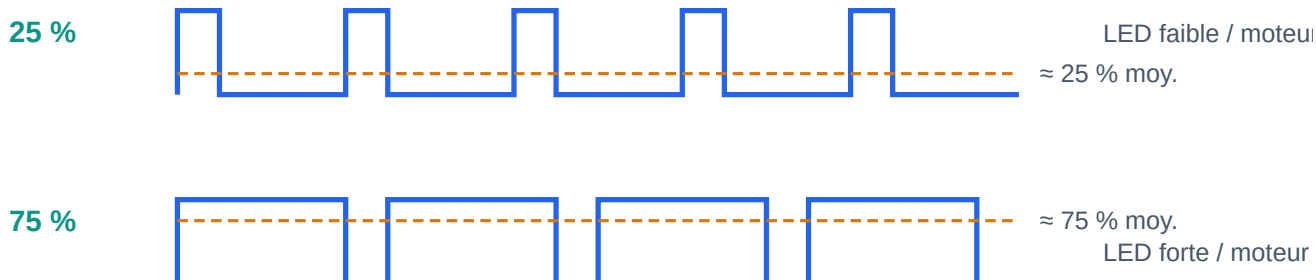
Compteurs matériels qui s'incrémentent à chaque tick d'horloge. Servent à : - mesurer le temps précisément, - générer des interruptions périodiques, - produire du **PWM** (voir ci-dessous), - compter des impulsions externes (codeur incrémental).

4.3 PWM (Pulse Width Modulation)

On fait varier la *puissance moyenne* en découpant un signal carré :

```
Rapport cyclique 25 % :  → LED faible, moteur lent
Rapport cyclique 75 % :  → LED forte, moteur rapide
```

PWM — la puissance moyenne varie avec le rapport cyclique



Même fréquence, seule la largeur d'impulsion change. L'œil et le moteur « intègrent » → luminosité / vitesse prop

PWM : le rapport cyclique fixe la puissance moyenne

Utilisé pour : luminosité de LED, vitesse de moteur, servo-moteurs (impulsion de 1 à 2 ms toutes les 20 ms), génération audio simple.

4.4 ADC / DAC

- **ADC** (Analog → Digital) : convertit une tension en nombre. Arduino Uno : 10 bits → 0–1023 pour 0–5 V. Résolution = $5 \text{ V} / 1024 \approx 4,9 \text{ mV}$.
- **DAC** (Digital → Analog) : l'inverse (rare sur les petits micros ; on émule avec du PWM + filtre RC).

4.5 Watchdog

Compteur qui redémarre le micro si le programme ne le « caresse » pas régulièrement. Filet de sécurité contre les plantages — obligatoire dans les produits industriels.

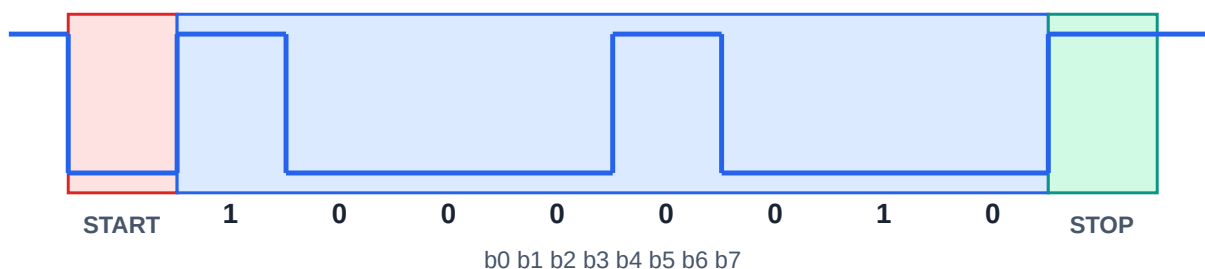
5. Les protocoles de communication

C'est LE sujet qui revient dans tous les entretiens embarqués.

5.1 UART (liaison série asynchrone)

- 2 fils : TX (émission) et RX (réception), croisés entre les deux équipements.
- Pas d'horloge partagée → les deux côtés doivent convenir d'un **baudrate** (9600, 115200 bauds...).
- Trame : bit de start, 8 bits de données, (parité), bit de stop.
- C'est ce qu'utilise `Serial.begin(9600)` sur Arduino.
- Variantes industrielles : **RS-232** ($\pm 12 \text{ V}$, point à point), **RS-485** (différentiel, multipoint, base du Modbus RTU — voir module 08).

Trame UART 8N1 — caractère 'A' = 0x41 = 0100 0001 (LSB d'abord)



Repos = état haut · 1 bit de start (bas) · 8 bits de données · 1 bit de stop (haut)

À 9600 bauds : 1 bit $\approx$ 104 μ s, trame $\approx$ 1,04 ms $\rightarrow$ 960 octets/s max

Trame UART 8N1 pour le caractère 'A'

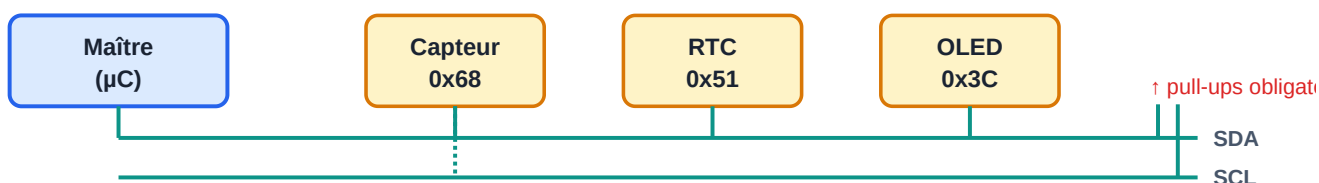
5.2 I2C (Inter-Integrated Circuit)

- 2 fils partagés par tous : **SDA** (données) + **SCL** (horloge), avec pull-ups.
- Un maître, plusieurs esclaves, chacun a une **adresse 7 bits** (ex. 0x68 pour le MPU-6050).
- Vitesse : 100 kHz / 400 kHz. Idéal pour capteurs, écrans OLED, RTC.
- Séquence : START $\rightarrow$ adresse + bit R/W $\rightarrow$ ACK $\rightarrow$ données $\rightarrow$ STOP.

5.3 SPI (Serial Peripheral Interface)

- 4 fils : **MOSI**, **MISO**, **SCK** (horloge), **CS** (sélection, un par esclave).
- Full duplex, très rapide (dizaines de MHz). Idéal pour cartes SD, écrans TFT, mémoires flash.
- Paramètres : polarité et phase d'horloge (modes 0 à 3).

I2C — 2 fils partagés (SDA + SCL), adressage 7 bits



SPI — full duplex, 1 fil CS par esclave (rapide : dizaines de MHz)



Topologies I2C (bus partagé) et SPI (un CS par esclave)

5.4 CAN (Controller Area Network)

- Bus différentiel 2 fils, multi-maître, très robuste. Standard de l'automobile et de beaucoup d'industriel.

- Trames courtes (8 octets classiques) avec identifiant qui sert de priorité (arbitrage sans collision).

5.5 Réseaux industriels (aperçu, détaillé aux modules 07-08)

- **Modbus RTU/TCP** : simple, vénérable, partout (Schneider en est l'origine).
- **PROFINET / PROFIBUS** : monde Siemens.
- **EtherCAT, CANopen, IO-Link** : motion et capteurs intelligents.

Comparatif rapide

	UART	I2C	SPI	CAN
Fils	2	2	4+	2
Vitesse	~1 Mbps	0,4 Mbps	50+ Mbps	1 Mbps
Multi-équipements	non	oui (adresses)	oui (1 CS chacun)	oui
Usage type	debug, GPS, modem	capteurs	écrans, SD	auto, industrie

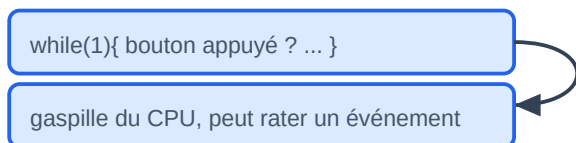
6. Interruptions et temps réel

6.1 Polling vs interruption

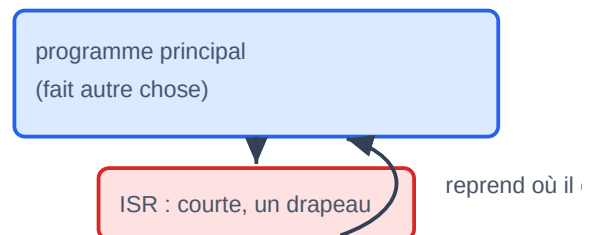
- **Polling** : la boucle principale vérifie sans cesse « le bouton est-il appuyé ? ». Simple, mais gaspille du CPU et peut rater des événements.
- **Interruption (IRQ)** : le matériel *suspend* le programme, exécute une fonction spéciale (**ISR**, Interrupt Service Routine), puis reprend où il en était.

Polling vs Interruption

Polling — la boucle DEMANDE sans cesse



Interruption — le matériel PRÉVIENT



Règle d'or : une ISR lève un drapeau (volatile !), stocke une valeur, et sort. Jamais de `delay()`, de `printf`, d'allocation.

Polling contre interruption

6.2 Règles d'or des ISR

1. **Courtes** : lever un drapeau, stocker une valeur, sortir. Jamais de `delay()`, de `printf`, d'allocation.
2. Les variables partagées entre ISR et boucle principale doivent être **volatile** (voir module 01) et, si > 1 octet, protégées (section critique : désactiver brièvement les interruptions).

3. Attention aux priorités : une ISR peut en interrompre une autre (nesting).

6.3 Temps réel

« Temps réel » ≠ « rapide ». Cela signifie **garantie de délai** : la réponse arrive *toujours* avant l'échéance. Un airbag qui répond en 1 s est rapide à l'échelle humaine mais inutile. D'où les **RTOS** (FreeRTOS, Zephyr — module 06) qui ordonnent des tâches par priorité de façon déterministe.

7. La chaîne de compilation (toolchain)

Que se passe-t-il entre ton `.c` et la puce qui clignote ?

```
main.c —[préprocesseur]—> main.i    (macros et #include expansés)
      —[compilateur]——> main.s    (assembleur)
      —[assembleur]——> main.o    (code machine, adresses non résolues)
      —[éditeur de liens]—> firmware.elf (tout assemblé, adresses fixées
                                         par le script de linkage .ld)
      —[objcopy]——> firmware.bin/.hex (image brute à flasher)
      —[programmeur]——> la puce (via USB, ST-Link, JTAG/SWD...)
```

Chaîne de compilation : du code source à la puce



Cross-compilation : on compile sur PC (x86) POUR une autre cible (ARM, AVR).
`arm-none-eabi-gcc` · `avr-gcc` · le programmeur écrit l'image dans la Flash.

La chaîne de compilation, du source à la puce

- **Cross-compilation** : on compile sur PC (x86) pour une autre cible (ARM) : `arm-none-eabi-gcc` , `avr-gcc` .
- **Débogueur matériel** (JTAG/SWD + GDB/OpenOCD) : points d'arrêt et inspection des variables sur la puce elle-même. Bien plus puissant que le `printf` de debug — mais apprend les deux.

8. Ce qu'il faut retenir

- Tout est binaire ; l'hexa est ta langue de lecture.
- Un microcontrôleur = CPU + mémoires + périphériques pilotés par registres.
- Écrire un bit dans un registre = agir sur le monde physique.
- UART / I2C / SPI / CAN : connais leurs fils, leurs forces, leurs usages.

- Interruptions courtes, variables partagées `volatile` .
- La toolchain transforme ton C en bits dans la flash.

➡ Passe au **Module 01 — Langage C** : c'est là que tout devient concret.